

HOL2: Backend Reliability Challenge – Resilient LLM Middleware Report

Goal: Build a resilient API-calling layer that survives transient failures under load by adding retries with jittered exponential backoff, simple caching for deterministic calls, and structured per-request logging including token usage and latency.

1. Middleware Script Implementation Plan

- **Core endpoint:** `LLMMiddleware.call(prompt, use_cache=True, ...)` wraps an LLM request end-to-end (timing, retries, caching, logging).
- **Retryable failures:** Detect and retry on 429 rate limit, timeouts, connection issues, and 5xx server errors.
- **Timeout handling:** Configure a request timeout (e.g., 30s) to prevent hung calls and free resources quickly.
- **Jittered exponential backoff:** Delay grows $1s \rightarrow 2s \rightarrow 4s$ (capped), plus random jitter ($\pm 200ms$) to avoid synchronized retry storms.
- **Cache:** In-memory dict keyed by a stable prompt hash; cache hits return instantly and log `cache_hit=true`.
- **Structured logging (JSONL):** One log line per request: timestamp, prompt hash/preview, model, latency_ms, retry_count, input/output tokens, cost_estimate, cache_hit, success, error.
- **Operational hooks:** Cache stats and cache clear functions; optional FastAPI wrapper for `/chat`, `/health`, `/cache`, `/logs`.

2. Retry Policy Explanation (100–150 words)

The middleware uses `max_retries=3` to balance reliability with bounded latency. On retryable failures (HTTP 429, timeouts, connection errors, and 5xx server responses), it waits with **exponential backoff** ($base_delay \times 2^{\text{attempt}}$: 1s, 2s, 4s, capped by `max_delay`) and adds a small **random jitter** ($\pm 200ms$). Jitter prevents many clients from retrying in lockstep, which reduces the risk of a thundering herd that can worsen rate limits. Retries stop after the final attempt or immediately for non-retryable errors (e.g., 4xx like 400/401/403). This policy avoids cascading failures by spreading load over time and failing fast when the request is invalid.

3. Performance Reflection (≈ 200 words)

Retries help when errors are **transient**: brief rate-limit bursts, flaky networks, or momentary 5xx issues. In these cases, a small number of delayed retries converts user-visible failures into successful responses, improving overall completion rate and reducing support incidents. Retries hurt when the failure is **persistent** (bad credentials, invalid requests, prolonged outages) because they add extra waiting time and can amplify load. That's why the middleware fails fast for non-retryable 4xx errors and caps the retry count.

Caching changes latency and cost dynamics dramatically for deterministic calls (`temperature=0`). A cache hit returns near-instantly and has effectively zero marginal token cost, which lowers p95 latency during traffic spikes and reduces API spend for repeated prompts. The trade-off is memory usage and the risk of serving stale outputs if prompts depend on changing context, so caching is best for stable FAQs and

idempotent requests.

p95 matters more than p50 because users remember the slow, failing tail. A middleware that trims tail failures produces a smoother experience: fewer timeouts, fewer confusing errors, and steadier response times under load.

4. Structured Log Fields (JSONL)

Field	Purpose
timestamp	Trace when the request occurred (ISO time).
prompt_hash / prompt_preview	Stable key for dedupe + human-readable hint.
model	Which model served the request.
latency_ms	End-to-end duration including retries.
retry_count	How many retries were needed.
input_tokens / output_tokens	Token usage for cost + capacity planning.
cost_estimate	Approx. USD cost (per-token pricing).
cache_hit	Whether response was served from cache.
success / error	Outcome + error message for debugging.

Deliverable check: middleware includes retry/backoff, timeout handling, optional caching, and structured logging.